

深圳大学实验报告

课程名称： Python程序设计

项目名称： 外星人入侵

学 院： 人工智能学院

专 业： 计算机科学与技术

指导教师： 樊超

报告人： 陈怡婷 学号： 2024440124

实验时间： 2025年11月19日

提交时间： 2025年11月19日

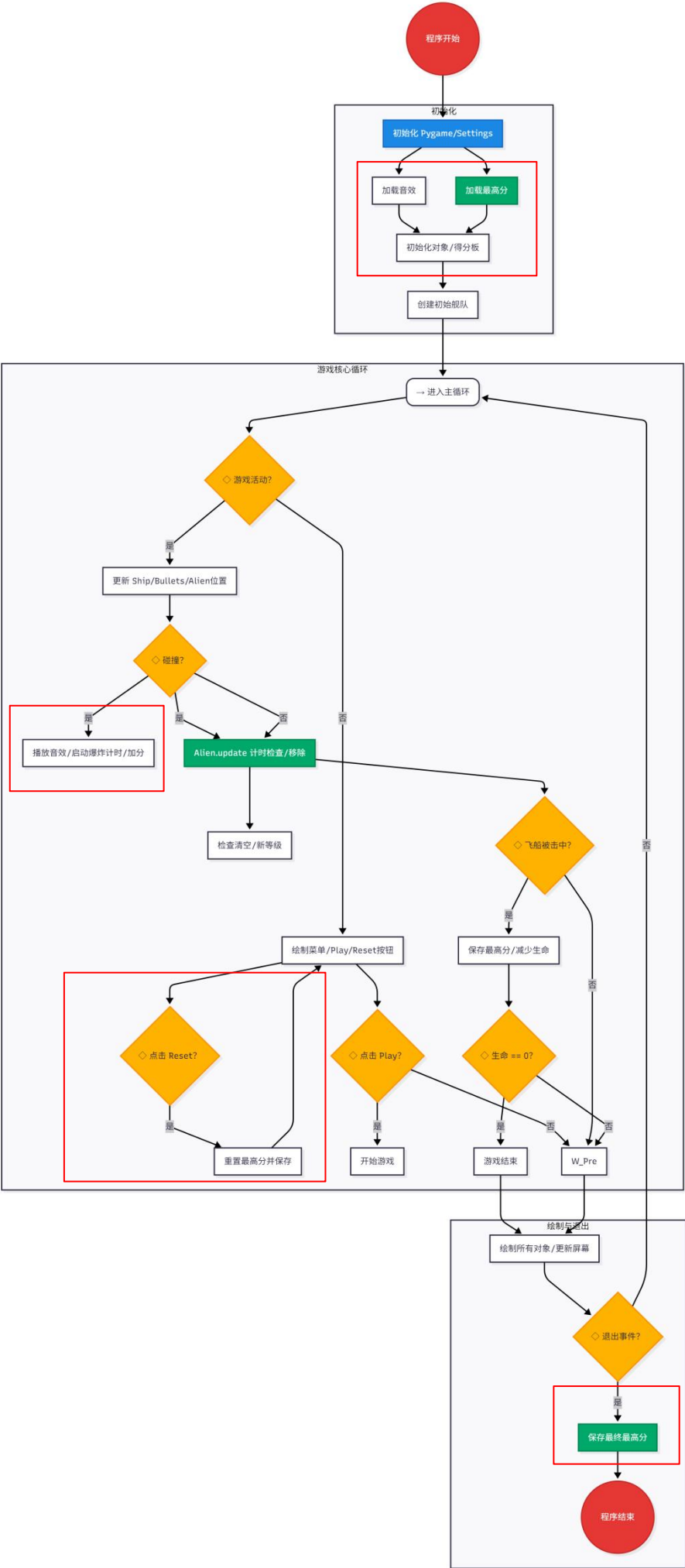


Figure1:Alien_invasion 流程图

1 Objectives

本次实验旨在通过对经典 Pygame 游戏《外星人入侵》进行功能扩展和优化，掌握游戏中的关键机制，并实践面向对象编程思想。具体目标包括：

1. 实现外星人爆炸动画与计时移除机制：通过引入爆炸状态和时间戳，实现外星人被击中后的爆炸效果和延迟移除，提升视觉反馈。
2. 集成游戏音效功能：添加射击声和爆炸声，通过 Pygame 混音器增强游戏的沉浸感和动态性。
3. 实现最高分持久化与重置功能：利用 json 模块实现最高分的保存与加载，确保游戏数据的持久性，并提供“重置最高分”的用户交互功能。

2 Overview

本实验通过对 Pygame 游戏的核心组件（alien.py, alien_invasion.py, game_stats.py）进行修改和扩展，实现了动画、音效和数据持久化三个主要方面的新功能。

Spot 1: 爆炸状态与生命周期管理: 在 Alien 类中引入 exploding 标志和 explosion_time 变量，控制外星人从被击中到真正从游戏中移除的生命周期。

Spot 2: 混音器与音效触发: 初始化 pygame.mixer，加载音效文件，并根据玩家发射子弹和外星人被击中的事件，在 _fire_bullet() 和 _check_bullet_alien_collisions() 中精确播放对应音效。

Spot 3: JSON 文件的数据持久化: 在 GameStats 类中实现 load_high_score() 和 save_high_score() 方法，利用 high_score.json 文件进行最高分的读写操作，并在游戏结束和用户点击“重置最高分”按钮时触发保存和重置逻辑。

3 Implementations

如Figure1所示，红框为我主要添加的功能：

Part 1: 外星人爆炸状态管理与计时移除 (alien.py & alien_invasion.py)

1. 状态定义与切换:

在 Alien.__init__ 中添加 self.exploding = False 和 self.explosion_time = 0.0。

在 _check_bullet_alien_collisions 方法中，当发生碰撞时，不再直接移除外星人，而是调用 alien.start_explosion() 方法，该方法会将 alien.image 切换为爆炸图像，并记录当前的毫秒时间戳 time.time() * 1000 到 self.explosion_time。

2. 计时与移除:

在 Alien.update() 中，新增逻辑：如果 self.exploding 为真，则计算当前时间与 self.explosion_time 的差值。若差值超过 self.settings.explosion_duration_ms (250ms)，则调用 self.kill() 将外星人移除。

3. 游戏逻辑保护:

在 _change_fleet_direction 中，增加了条件判断 if not alien.exploding:，确保只有未爆炸的外星人才会参与舰队下落和转向，避免爆炸中的外星人影响游戏节奏。

Part 2: 游戏音效集成 (alien_invasion.py)

混音器初始化与加载: 在 AlienInvasion.__init__ 中调用 pygame.mixer.init() 进行初始化，并实现 _load_sounds() 方法加载音效文件 (laser1.wav 和 DeathFlash.flac)。

4. 音效触发:

射击音效: 在 _fire_bullet() 方法中，子弹实例化并添加到 Group 后，立即调用 self.sound_shoot.play()。

爆炸音效: 在 _check_bullet_alien_collisions 中，当外星人被标记为爆炸状态时，调用 self.sound_explosion.play()。

Part 3: 最高分持久化与重置 (game_stats.py)

5. 数据持久化:

GameStats.load_high_score(): 检查 high_score.json 文件是否存在。如果存在，尝试使用 json.load() 读取最高分；如果文件不存在或读取时发生 JSONDecodeError，则安全地返回 0。

GameStats.save_high_score(): 使用 json.dump() 将当前的 self.high_score 写入文件，确保在游戏退出或结束时数据得到保存。

6. 重置功能:

GameStats.reset_high_score(): 将 self.high_score 设为 0，并立即调用 self.save_high_score() 写入文件，同时更新得分板显示。

alien_invasion.py 中实例化了 "Reset High Score" 按钮，并通过 _check_reset_score_button 方法响应点击事件，调用上述重置逻辑。

4 Results

具体代码和实验细节已上传Github，Github网址：https://github.com/chenyitingkitty20051126-lgtm/Alien_invasion

游戏页面如Figure2所示：

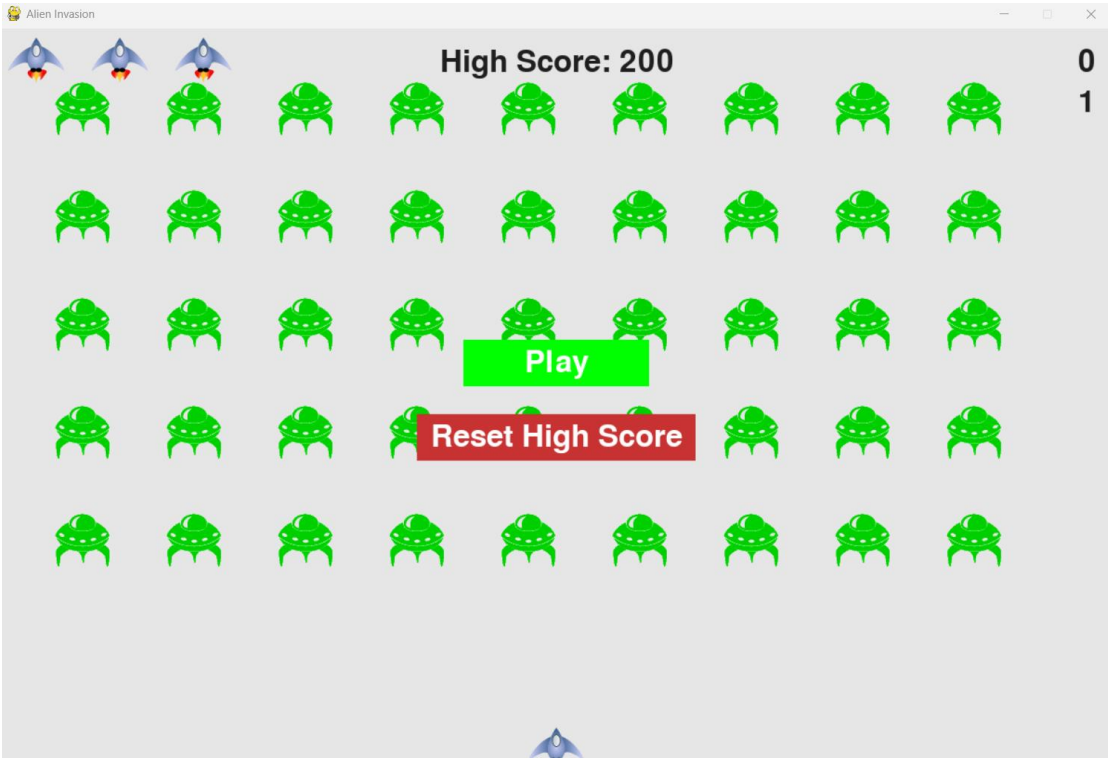


Figure2

测试结果如表 1所示：

测试场景编号	场景描述	预期结果	实际结果
TC-A1	子弹击中外星人	外星人切换为爆炸图像，播放爆炸音效，得分增加。外星人不会立即消失。	符合
TC-A2	爆炸计时结束	爆炸图像持续 250ms 后，外星人从屏幕上彻底移除 (self.kill())。	符合
TC-B1	游戏最高分更新与保存	当前得分超过历史最高分时，得分板更新，游戏退出后重新启动，新最高分被正确加载。	符合（加载 high_score.json 中保存的值）
TC-B2	“重置最高分”按钮	在主菜单点击该按钮后，最高分显示为 0，并持久化到文件。	符合
TC-C1	舰队移动中爆炸	爆炸中的外星人在舰队整体下落和水平转向时，位置保持不变。	符合（通过 if not alien.exploding: 保护）

表 1

5 Discussion

本次实验在保持游戏基础功能稳定的前提下，成功扩展了动画、音效和数据持久化功能，极大地提高了游戏的用户体验和完整性。

音效兼容性问题分析 (Inefficiencies): 在 alien_invasion.py 的 _load_sounds() 方法中，对 DeathFlash.flac 文件的加载使用了 try...except 块。虽然代码提供了占位逻辑，但 .flac 格式的音频文件在某些 Pygame 版本或缺乏系统解码器的环境中可能无法播放，这引入了潜在的兼容性风险。建议在实际项目中，将音效文件预处理为通用格式（如 .wav）以保证跨平台稳定性。

爆炸动画优化方向 (Improvement): 当前爆炸动画是静态图片+计时移除，缺乏动态感。若要达到更真实的爆炸效果，需要将 alien.py 中的 self.image_explosion 替换为一个包含多帧爆炸画面的图像序列 (Sprite Sheet)。Alien.update() 方法需要修改为：在爆炸状态下，根据当前时间

`current_time_ms` 和总爆炸时长 `self.settings.explosion_duration_ms`，计算当前应该显示的帧序号，然后动态切割和更新 `self.image`，实现流畅的动画效果。

代码职责分离 (Coupling): `alien_invasion.py` 中的 `_check_bullet_alien_collisions` 方法同时处理了碰撞、得分增加、爆炸状态切换和爆炸音效播放等多个任务。这使得 `AlienInvasion` 类承担了过多的游戏逻辑。为了更好地遵循单一职责原则，可以将得分更新和音效播放逻辑通过自定义事件或回调函数机制，从碰撞检测的核心循环中分离出来，从而提高代码的可维护性和模块化程度。

指导教师批阅意见：

成绩评定：

指导教师签字：

备注：

- 报告内的项目或内容设置，可根据实际情况加以调整和补充。
- 教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。